

**SISTEMAS DE COMPUTADORES – 2024/2025**  
**Exame Época Normal**

Versão A

**Apenas autorizada a consulta da folha oficial**  
**Duração: 2h**

Nome: \_\_\_\_\_ Nº \_\_\_\_\_

**Grupo I**  
**(8/20 valores)**

**NOTAS:**

- 1. Em todas as questões deverá assinalar apenas uma resposta.**
- 2. Se a resposta assinalada for incorreta sofrerá uma penalização de 1/3 da cotação da pergunta.**
- 3. Devem ser entregues todas as folhas do exame.**

**1** – Em Linux, que mecanismo de gestão de memória permite ao sistema operativo partilhar páginas entre o processo pai e o processo filho até que algum deles escreva nessa página de forma a diminuir o custo de criação de processos?

- a) Mapeamento direto de memória
- b) *Copy-on-write*
- c) *Shadow paging*
- d) Duplicação imediata (*eager copy*)

**2** – Em Linux, quando um processo termina e o seu pai não invoca a função *wait()* nem *waitpid()*, como é apresentado o estado desse processo pelo comando *ps*?

- a) Suspenso (*suspended*)
- b) *Zombie*
- c) Bloqueado (*blocked*)
- d) Órfão (*orphan*)

**3** – Qual a sequência correta de ações quando ocorre uma falta de página (*page fault*) num sistema de memória virtual?

- a) A MMU coloca a página em cache, reinicia a instrução e só depois lança a exceção
- b) A MMU lança uma exceção de *page fault*, o *kernel* seleciona uma página vítima, carrega a página em memória e, por fim, reinicia a instrução que causou a falta
- c) O *kernel* envia um sinal SIGSEGV ao processo, copia a página para o disco e termina o processo
- d) A MMU aguarda até que o processo liberte memória, sem gerar qualquer interrupção

4 – Em sistemas com paginação, qual o propósito principal da *Translation Lookaside Buffer* (TLB)?

- a) Armazenar páginas não referenciadas no disco para libertar memória física
- b) Traduzir o endereço virtual diretamente em deslocamento dentro da tabela de páginas
- c) Atuar como uma cache de entradas da página, acelerando a tradução de endereços virtuais em físicos
- d) Manter a lista de páginas elegíveis para serem trocadas em cenários de falta de memória física

5 – Qual das seguintes afirmações sobre os mecanismos de comunicação entre processos (IPC) está correta?

- a) A passagem de mensagens é tipicamente mais rápida e flexível do que a memória partilhada, pois não requer intervenção do *kernel* do sistema operativo
- b) A memória partilhada é mais fácil de implementar em sistemas distribuídos, mas é menos eficiente do que a passagem de mensagens para a troca de grandes volumes de dados
- c) O acesso simultâneo à memória partilhada requer estratégias de sincronização para garantir a coerência dos dados
- d) A passagem de mensagens é ideal para a troca de grandes quantidades de dados, pois todas as operações são realizadas no espaço do utilizador sem chamadas de sistema

6 – Relativamente aos semáforos e às suas operações atômicas *down()* e *up()*, qual das seguintes afirmações é verdadeira?

- a) Um semáforo com valor inicial 0 é tipicamente usado para exclusão mútua no acesso a recursos partilhados
- b) A operação *down(s)* incrementa o valor do semáforo *s* e bloqueia o processo se *s* tiver o valor 0
- c) As operações *down()* e *up()* são garantidas como indivisíveis pelo *kernel* do sistema operativo, prevenindo condições de corrida na manipulação do semáforo
- d) Um semáforo de contagem (com valor inicial maior que 0) é usado exclusivamente para sinalizar a ocorrência de um evento num algoritmo de barreira

7 – Considere as estratégias de bloqueio *coarse-grained* e *fine-grained*. Qual das seguintes afirmações descreve corretamente o compromisso associado a estas abordagens?

- a) O bloqueio *coarse-grained* oferece melhor escalabilidade e concorrência, mas é mais complexo de implementar corretamente
- b) É mais fácil de provar a correção do bloqueio *fine-grained*, mas leva a um desempenho inferior em cenários de baixa contenção
- c) O bloqueio *coarse-grained* é mais simples de escrever e provar a sua correção, mas pode resultar em baixo desempenho em alta contenção devido à limitada concorrência
- d) O bloqueio *fine-grained* tem um *overhead* menor devido à gestão de bloqueios, o que o torna ideal para todas as aplicações concorrentes

**8** – Em que cenário os *read-write locks* (como *pthread\_rwlock\_t*) são particularmente úteis?

- a) Em aplicações onde a maioria das operações envolve a modificação frequente de dados compartilhados por múltiplos escritores
- b) Em sistemas com poucos acessos a dados compartilhados, onde o *overhead* de gestão de bloqueios é irrelevante
- c) Em qualquer aplicação concorrente, independentemente do padrão de acesso, devido à sua simplicidade de implementação
- d) Em aplicações com um número significativamente maior de leitores do que escritores, onde a concorrência de leitura é crucial

**9** – Um processo *I/O-bound* e um processo *CPU-bound* estão a ser executados num sistema operativo. Qual das seguintes afirmações descreve mais precisamente o comportamento típico destes processos e as suas implicações no escalonamento?

- a) Processos *I/O-bound* tendem a monopolizar a CPU por longos períodos, enquanto processos *CPU-bound* frequentemente bloqueiam para operações de *input/output*
- b) Processos *CPU-bound* são mais adequados para sistemas não-preemptivos, pois libertam a CPU regularmente, otimizando o débito computacional
- c) A distinção entre *I/O-bound* e *CPU-bound* é irrelevante para a escolha do algoritmo de escalonamento, pois todos os processos são tratados de forma idêntica
- d) Processos *I/O-bound* passam a maior parte do tempo à espera de *input/output* e executam por curtos períodos, o que os torna mais sensíveis ao tempo de resposta do escalonador

**10** – Um sistema de tempo real utiliza escalonamento baseado em prioridades e partilha recursos entre processos. Um processo de alta prioridade (*P\_alta*) tenta aceder a um recurso que está bloqueado por um processo de baixa prioridade (*P\_baixa*). Qual das seguintes opções descreve corretamente o problema que pode surgir e uma solução para o mitigar?

- a) O problema é a inanição (*starvation*) de *P\_alta*, e a solução é o envelhecimento de *P\_baixa*
- b) O problema é o bloqueio permanente (*deadlock*) entre *P\_alta* e *P\_baixa*, resolvido pelo Protocolo de Teto de Prioridade que previne a partilha de recursos
- c) O problema é o *overhead* excessivo de comutação de contexto, e a solução é aumentar o *time slice* de *P\_baixa*
- d) O problema é a inversão de prioridades, onde *P\_alta* é indiretamente bloqueado por *P\_baixa*; o Protocolo de Herança de Prioridade pode mitigar isso

## Grupo II (12/20 valores)

Versão A

### NOTAS:

1. Responder cada questão numa folha separada, devidamente identificada.
2. Indicar a versão do exame em cada uma das folhas.
3. Não é necessário indicar o nome das bibliotecas usadas na resolução.

[5 valores] 1) Implemente um programa em C que aplique um padrão *master-slave* simples usando processos e *pipes*. O objetivo é ler do *stdin* uma sequência de M números inteiros não negativos, distribuí-los por um número fixo de N processos *slave*, fazer com que cada *slave* calcule o fatorial dos valores que lhe forem atribuídos e, de seguida, enviar os resultados de volta ao processo *master*, que os **imprime pela ordem em que foram lidos do *stdin***.

A sua solução deve garantir a criação do número adequado de processos e de *pipes*, permitindo que os processos *slave* sejam **executados de forma concorrente**.

[7 valores] 2) Implemente um programa em C que crie três *threads*: duas delas simulam sensores a gerar leituras de temperatura e a terceira serve de registo, consumindo essas leituras e apresentando-as no ecrã.

As *threads* comunicam através de um **buffer circular** com capacidade para 10 valores inteiros (cada valor representa uma leitura de sensor).

Cada sensor gera valores incrementais (por exemplo, 15, 16, 17, ...) e coloca-os no *buffer*; a *thread* registo lê cada valor e imprime algo como:

```
Leitura 3 do Sensor A: 17
Leitura 2 do Sensor B: 23
...
```

Assume-se que são trocadas apenas 40 leituras de cada sensor ao longo da execução do programa.

Garanta que:

- Se o *buffer* estiver cheio, as *threads* sensores ficam **bloqueadas de forma passiva** até haver espaço livre;
- Qualquer acesso ao *buffer* (leitura ou escrita) ocorre sob **exclusão mútua**;
- Se o *buffer* estiver vazio, a *thread* registo fica **bloqueada de forma passiva** até surgir nova leitura.